



Maschinelles Lernen

6. Einführung neuronale Netze

Prof. Dr. Dennis Janka

Überwachtes Lernen = Suchen von Funktionen

Motivation:

Aufgaben, die sich als **funktionaler Zusammenhang** formulieren lassen, z.B.:

- Historische Verkaufszahlen $\rightarrow$ zukünftige Verkaufszahlen
- Email $\rightarrow$ Spam / nicht-Spam
- Liste von Wörtern $\rightarrow$ nächstes Wort
- Allgemein: Input $\rightarrow$ Output, $X \rightarrow Y$ (Feature $\rightarrow$ Label)

Voraussetzung: Input und Output müssen als **Vektoren** vorliegen.

(Überwachtes) Lernen = **Suchen einer mathematischen Funktion**, die diesen Zusammenhang möglichst gut beschreibt.

... durch Minimieren einer Loss-Funktion

- Das „Suchen einer Funktion“ wird geleitet von einer **Loss-Funktion**
- Aufgabe der Verlustfunktion:

Quantifizierung, wie gut ein Modell f (repräsentiert durch Koeffizienten $\mathbf{w}$) die Trainingsdaten beschreibt

Beispiele Loss-Funktion

- Mean squared error (bei linearer Regression):

$$L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (y_i - f(\mathbf{x}_i; \mathbf{w}))^2$$

- Binäre Kreuzentropie (bei logistischer Regression) :

$$L(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N y_i \log f(\mathbf{x}_i; \mathbf{w}) + (1 - y_i) \log (1 - f(\mathbf{x}_i; \mathbf{w}))$$

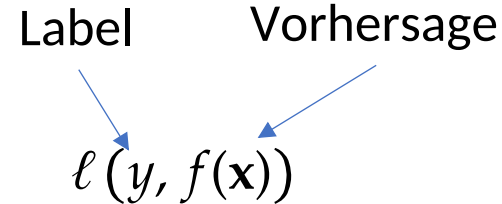
- Kreuzentropie (bei multinomialer logistischer Regression):

$$L(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K (y_i)_k \log f(\mathbf{x}_i; \mathbf{w})$$

—> Alle Loss-Funktionen sind Mittelwert des Fehlers einzelner Datenpunkte (Summe über 1 bis N)

Loss-Funktion allgemein

- Eine Loss-Funktion für einen Punkt



quantifiziert den „**Loss**“ („**Verlust**“) / die **Kosten**, wenn für ein Input-Beispiel $\mathbf{x}$ die Vorhersage $f(\mathbf{x})$ gemacht wird und der wahre Wert (das Label) y ist.

- Beispiel binäre Kreuzentropie:

$$\ell(y, f(\mathbf{x})) = -(y \log f(\mathbf{x}) + (1-y) \log(1-f(\mathbf{x})))$$

- Beispiel quadratischer Fehler:

$$\ell(y, f(\mathbf{x})) = (y - f(\mathbf{x}))^2$$

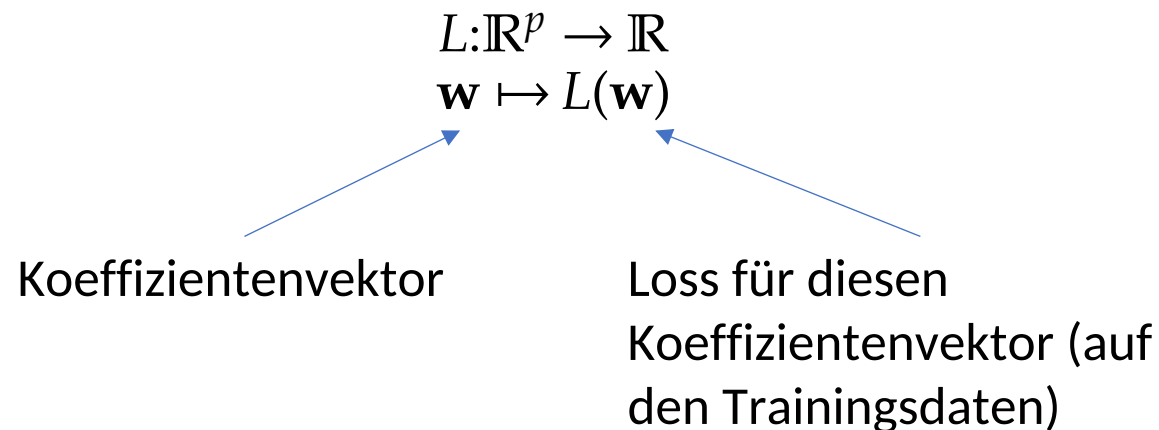
Loss-Funktion auf Trainingsdaten

- Allgemein können wir unseren Loss auf den Trainingsdaten schreiben als:

$$L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \ell(y_i, f(\mathbf{x}_i; \mathbf{w}))$$

wobei $f(\mathbf{x}_i; \mathbf{w})$ bedeutet, dass die Vorhersage nicht nur vom Feature-Vektor $\mathbf{x}_i$ abhängt, sondern auch von den Koeffizienten $\mathbf{w}$

- Wir betrachten die Loss-Funktion auf den Trainingsdaten als multivariate Funktion:



Ziel des Lernens

- Eigentliches Ziel des Lernens: Gute Vorhersagen auf *unbekannten* Daten machen
—> mathematische Formulierung?
- Ziel ist es, eine Funktion zu finden, die den **Generalisierungsfehler** (auch: **erwarteter Verlust** oder **Risiko** eines Modells) über alle *möglichen* Punkte $(\mathbf{x}, y)$ (gewichtet mit ihrer Wahrscheinlichkeit) minimiert:

$$\min_{\mathbf{w}} \mathbb{E} [\ell(y, f(\mathbf{x}; \mathbf{w}))]$$

Erwartungswert des Verlustes über die gesamte *Wahrscheinlichkeitsverteilung* aller Paare $(\mathbf{x}, y)$

- Diesen Erwartungswert kann man nicht berechnen, da „Wahrscheinlichkeitsverteilung aller Paare“ $(\mathbf{x}, y)$ nicht bekannt ist

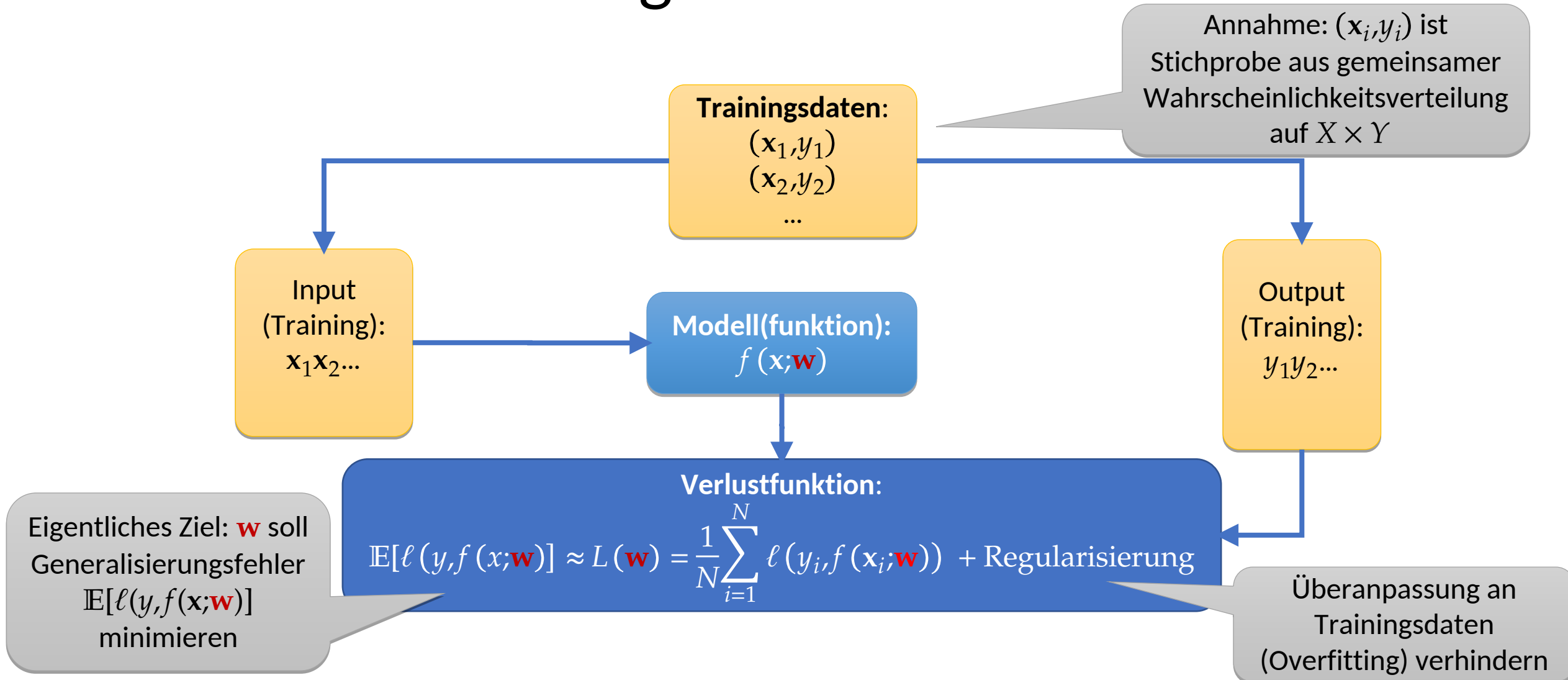
Ziel des Lernens

- Schätzung für den Erwartungswert: Mittelwert einer Stichprobe – unsere Trainingsdaten!
- Man macht folgende Approximation:

$$L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \ell(y_i, f(\mathbf{x}_i; \mathbf{w})) \approx \mathbb{E}[\ell(y, f(\mathbf{x}; \mathbf{w}))]$$

- Minimieren von $L(\mathbf{w})$ für gegebene Daten wird auch **empirische Risikominimierung** genannt.
- Mit einem Testdatensatz simulieren wir „alle unbekannten Punkte“, um sicherzugehen, dass wir bei der empirische Risikominimierung nicht nur die spezifischen Muster der Trainingsdaten beschreiben (Overfitting).

Zusammenfassung Überwachtes Lernen



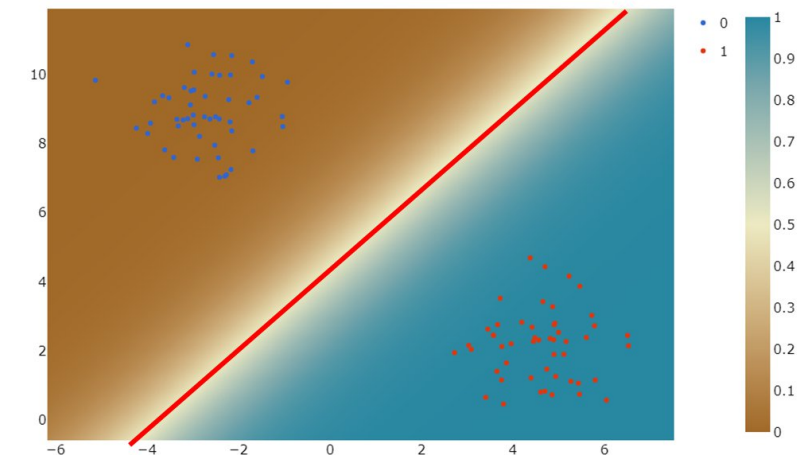
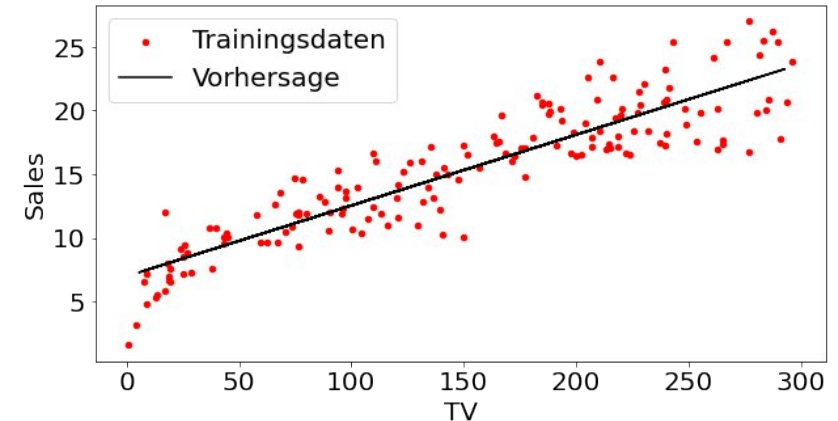
Zentrale Fragen für die nächsten Wochen

1. Wie sehen gute Modelle, d.h. Funktionen f , aus?
2. Wie trainiert man die Modelle?
 - a) Wie findet man das Minimum von $L(\mathbf{w})$?
 - b) Wie stellt man sicher, dass $L(\mathbf{w})$ gute Approximation für $\mathbb{E}[\ell(y, f(\mathbf{x}; \mathbf{w}))]$ ist?

Überwachtes Lernen: Modelle bisher

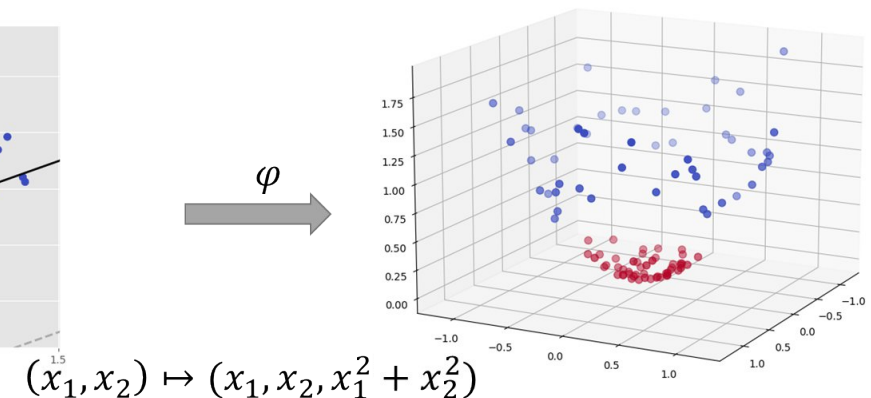
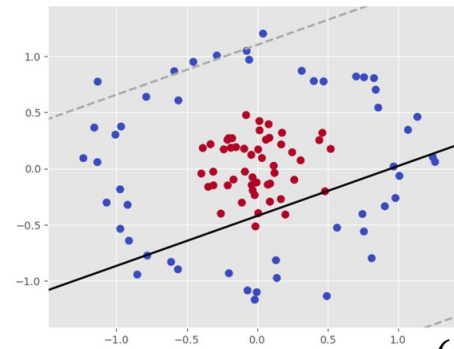
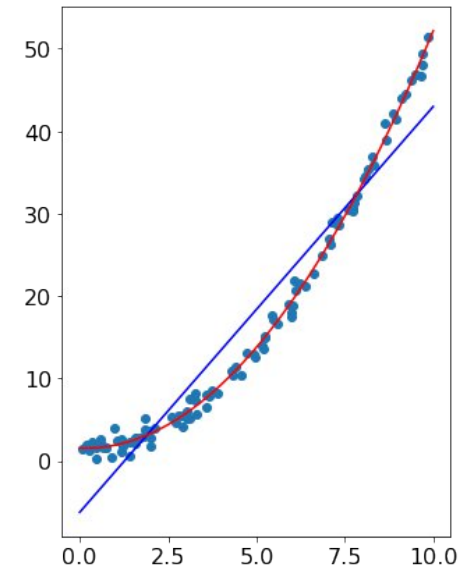
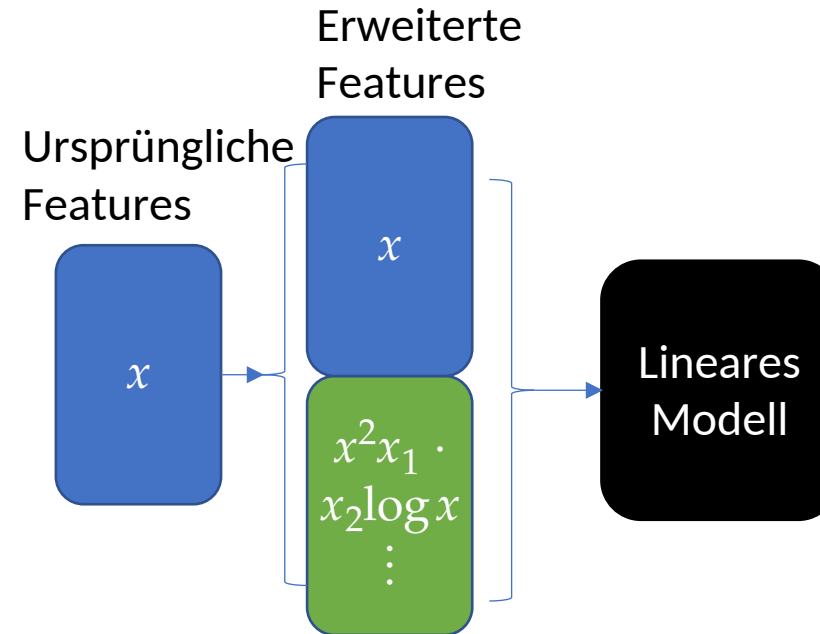
- Wir haben bisher zwei einfache Typen von Vorhersagemodellen (Modellfunktionen) kennengelernt:
 - Lineare Regression um numerische Vorhersagen zu machen
 - Logistische Regression um diskrete Vorhersagen zu machen bzw. Objekte zu klassifizieren
- Beides sind **lineare** Modelle, da die Vorhersage (bzw. decision boundary bei Klassifizierung) auf einer linearen Funktion (Gewichtung) der vorhandenen Features beruht, $\mathbf{w}^T \mathbf{x}$ bzw. $\sigma(\mathbf{w}^T \mathbf{x})$.

Lineare Modelle



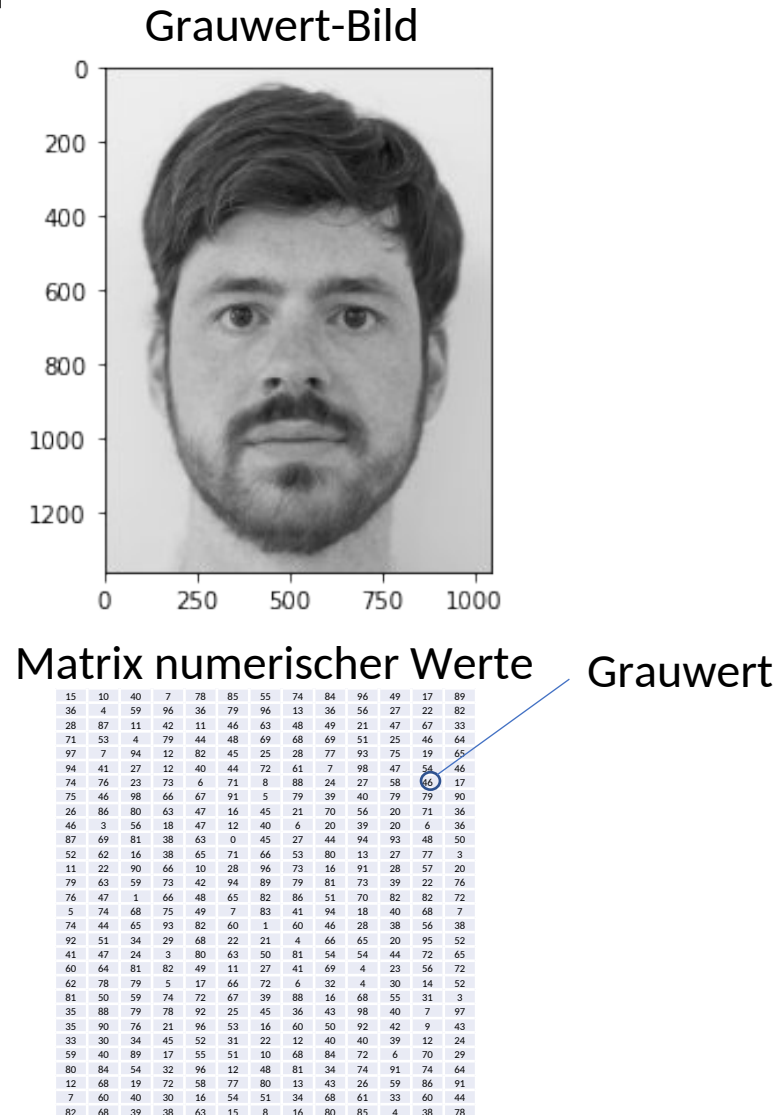
Erweiterungen auf Nichtlinearitäten

- Durch händisches Einführen neuer Features ist es möglich, ein lineares Modell zu einem nichtlinearen Modell zu erweitern
- Problem: Man muss vorher die Art der Nichtlinearität festlegen
- Problem: „Naheliegende“ Nichtlinearität
en wie Polynome
generalisieren oft schlecht
(neigen zu Overfitting)



Anwendung: Maschinelles Sehen

- Beispiel: Person männlich oder weiblich? (binäre Klassifikation)
 - Ein Bild der Auflösung 1280×1024 Pixel wird als Featurevektor im $\mathbb{R}^{1310720}$ dargestellt!
 - Features sind stark korreliert, einzelne Features haben sehr geringe Aussagekraft
- > „händisch“ erstellte, „einfache“ nichtlineare Features, auf die ein lineares Modell aufsetzen kann, führen nicht zum Ziel



Allgemeine Nichtlineare Modelle: Anforderungen

Gesucht:

- Generische, nichtlineare Modellfunktionen: neue (abgeleitete) „Features“ sollen möglichst automatisch gelernt werden
- Für Lernen geeignet (differenzierbar nach Parametern)
- Aus Bausteinen aufbaubar (brauche auch geeignete Notation zur Spezifikation solcher Funktionen)
(bisher haben wir Funktionen als geschlossene algebraische Ausdrücke geschrieben,
z.B. $f(\mathbf{x}) = w_0 + w_1 x_1 + w_2 x_2^2$)

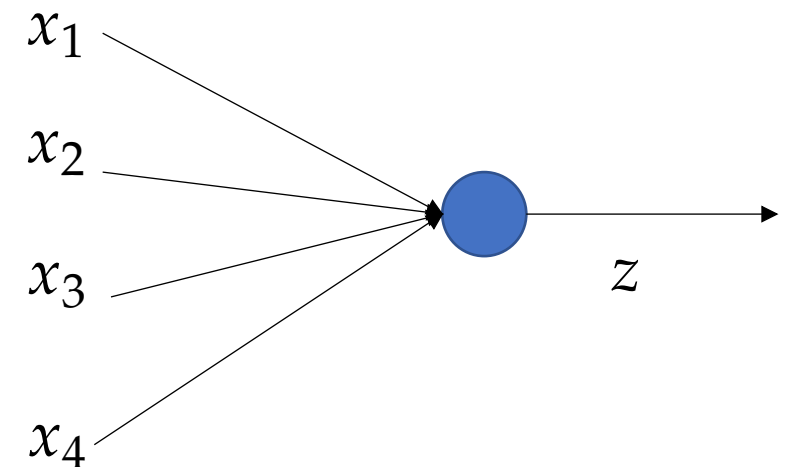
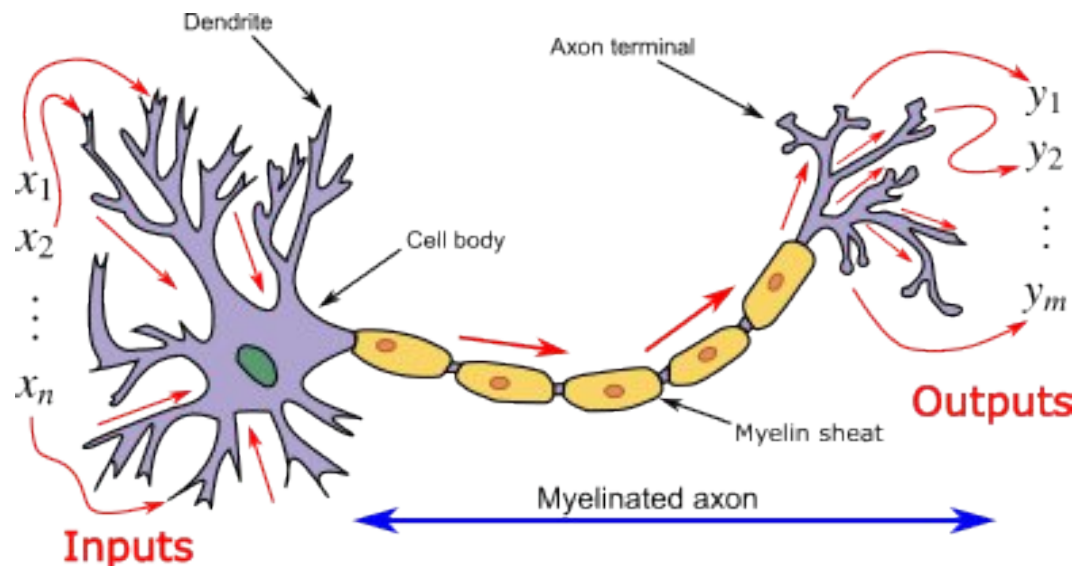
Einführung neuronale Netze

Künstliches Neuron (Perzeptron): Inspiration aus der Biologie

- Vereinfachtes Modell: An Nervenzellen (Neuronen) liegen n Eingangssignale an
- Neuron “feuert”, wenn Schwelle für Spannung überschritten wird und kann so Information weitergeben:

$$z = \begin{cases} 1, & \text{wenn } w_1x_1 + \dots + w_nx_n + b \geq 0 \\ 0, & \text{sonst} \end{cases} = : \text{threshold}(\mathbf{w}^T \mathbf{x} + b) \in \{0,1\}$$

- Lernen = Anpassen der Gewichtungen der Eingänge w_i und des Schwellwerts $-b$.

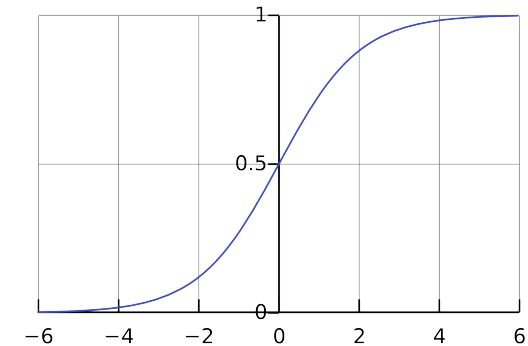
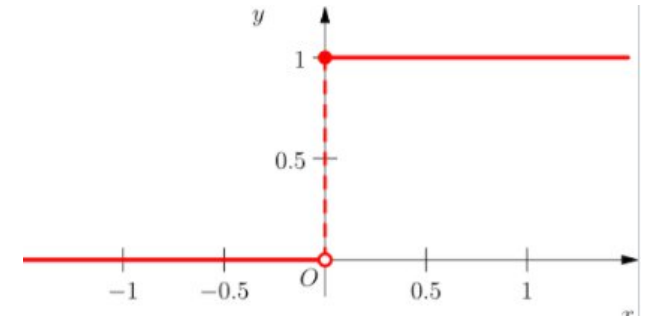
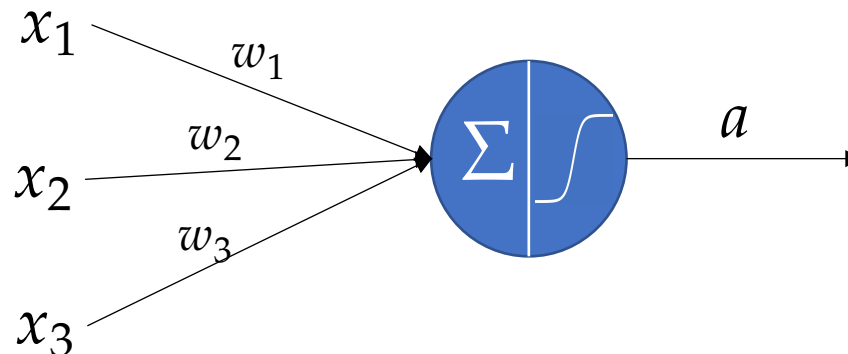


Neuron mit Sigmoid-Funktion

- Problem: Die Funktion $\text{threshold}(\mathbf{w}^T \mathbf{x} + b)$ ist nicht differenzierbar (Heaviside-Funktion)
- Idee (zunächst): Benutze Sigmoid-Funktion, wie für die logistische Regression:

$$\sigma(\mathbf{w}^T \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)'}}$$

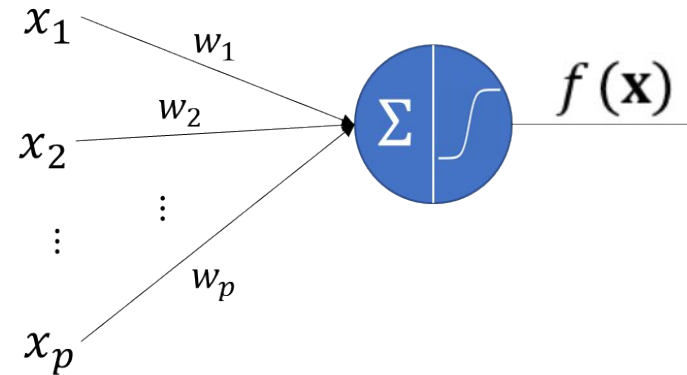
- Bildlich zusammengefasst (Bsp. mit drei Features):



Lineare Regression und logistische Regression in „Neuronen-Schreibweise“

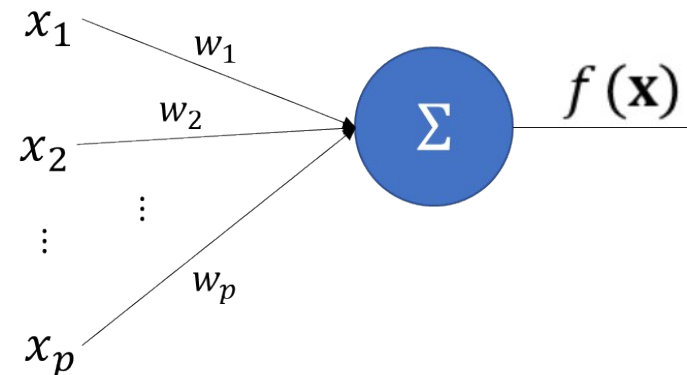
Modellfunktion logistische Regression:

$$f(\mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x})}} =$$



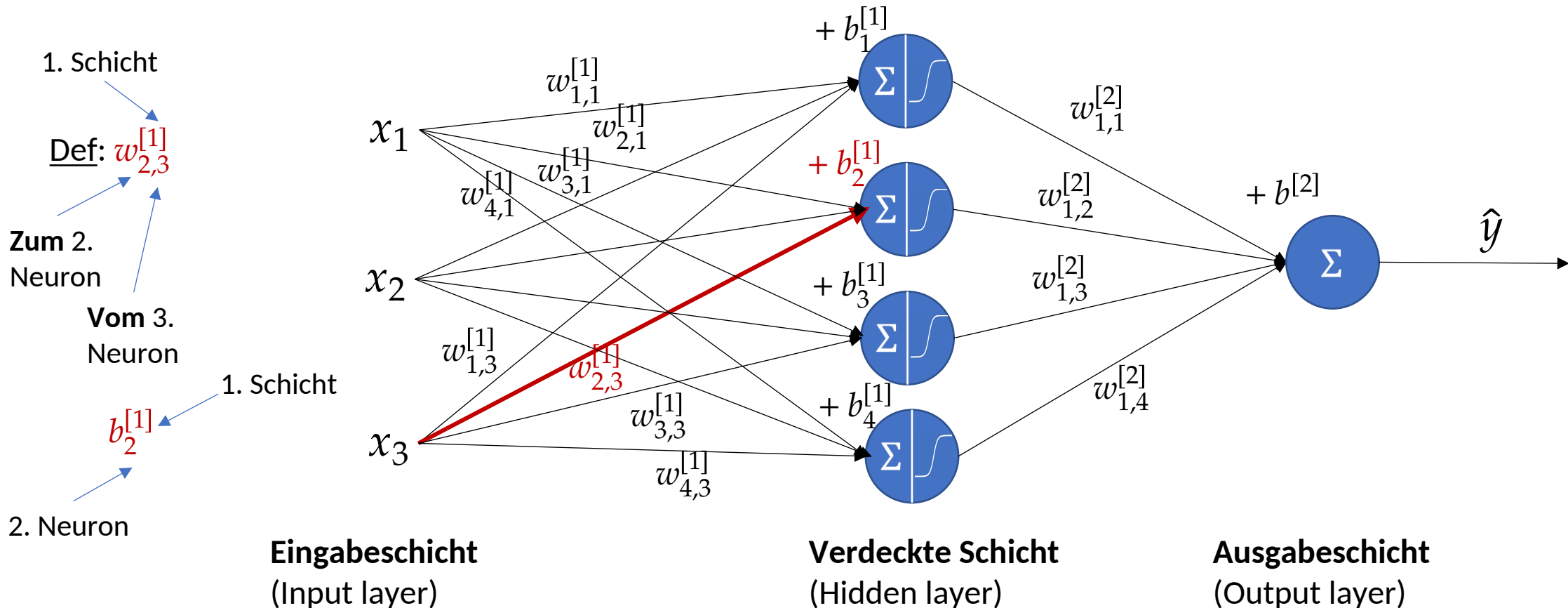
Modellfunktion lineare Regression:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} =$$



Neuronales Netz mit einer Schicht

Idee: Instantiiere mehrerer solcher Neuronen und ordne sie in einer Schicht an. Der Output $\hat{y}$ (=Funktionswert) ist gewichtete Summe der Outputs (=Aktivierungen) der einzelnen Neuronen.

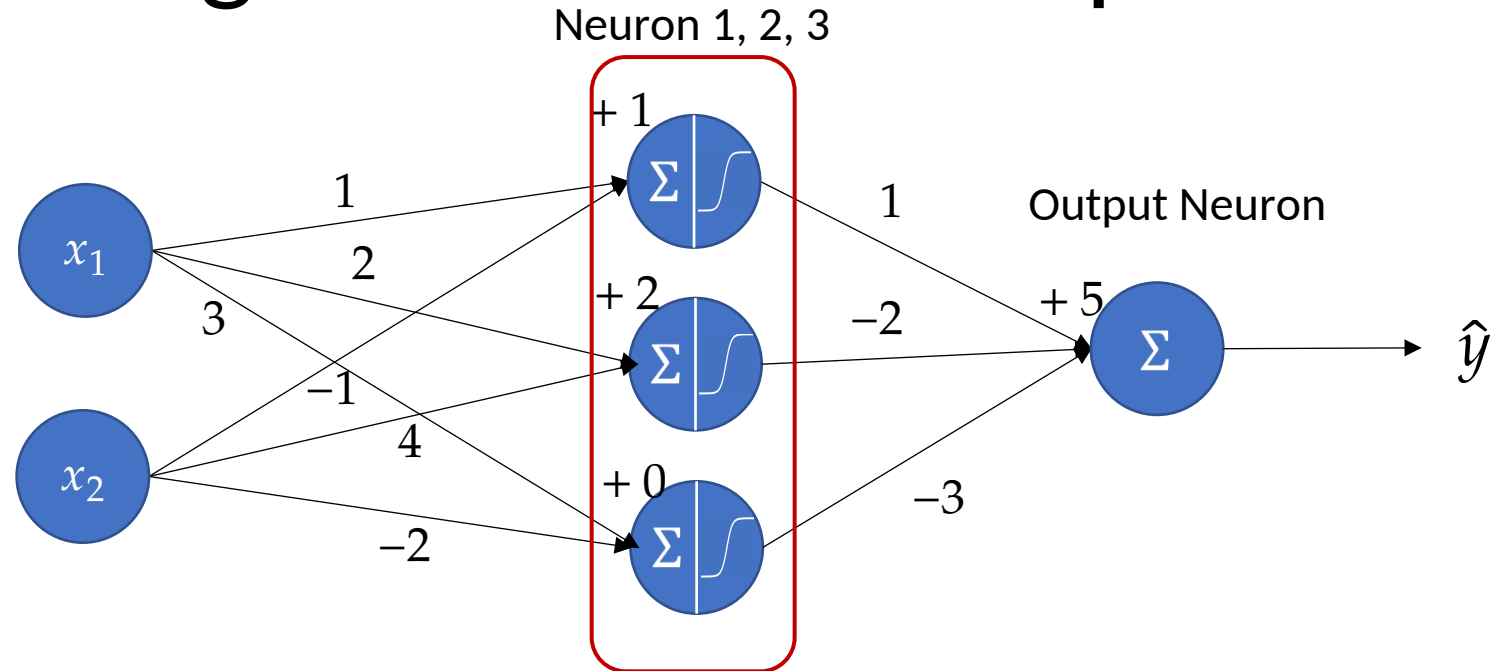


Beispiel: Vorhersage für einen Datenpunkt

Gewichte:

$$\begin{aligned} w_{1,1}^{[1]} &= 1 \\ w_{2,1}^{[1]} &= 2 \\ w_{3,1}^{[1]} &= 3 \\ w_{1,2}^{[1]} &= -1 \\ w_{2,2}^{[1]} &= 4 \\ w_{3,2}^{[1]} &= -2 \\ b_1^{[1]} &= 2 \\ b_2^{[1]} &= 0 \end{aligned}$$

$$\begin{aligned} w_{1,1}^{[2]} &= 1 \\ w_{1,2}^{[2]} &= -2 \\ w_{1,3}^{[3]} &= -3 \\ b^{[2]} &= 5 \end{aligned}$$



Vorhersage für Punkt $(x_1, x_2) = (1, 2)$:

$$\text{Output (Aktivierung) Neuron 1: } a_1 = \sigma(1 \cdot 1 - 1 \cdot 2 + 1) = \frac{1}{1+e^0} = 0.5$$

$$\text{Output (Aktivierung) Neuron 2: } a_2 = \sigma(2 \cdot 1 + 4 \cdot 2 + 2) = \frac{1}{1+e^{-12}} = 0.99$$

$$\text{Output (Aktivierung) Neuron 3: } a_3 = \sigma(3 \cdot 1 - 2 \cdot 2 + 0) = \frac{1}{1+e^1} = 0.27$$

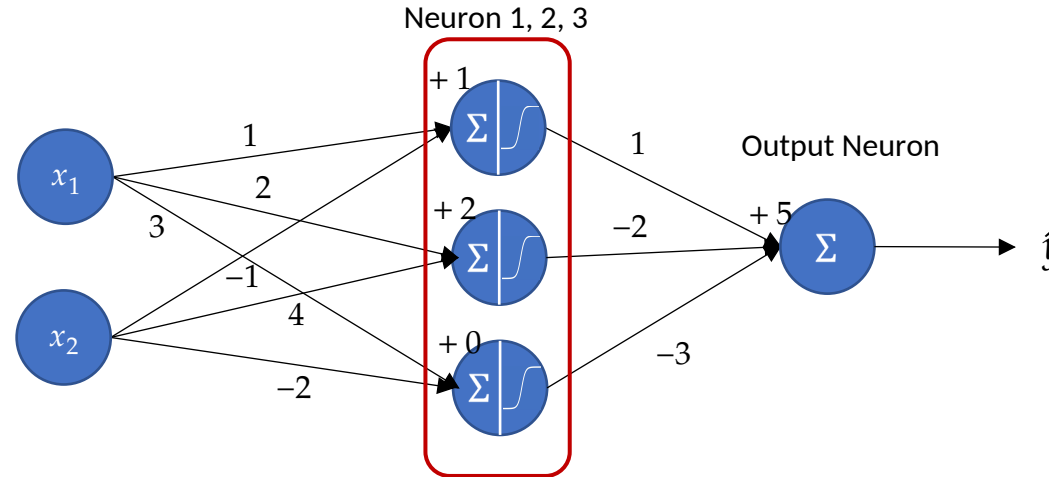
$$\text{Output Neuron: } a_{out} = 1 \cdot 0.5 - 2 \cdot 0.99 - 3 \cdot 0.27 + 5 = 1.312 = \hat{y}$$

Neuronales Netz: Matrixschreibweise

Anmerkung: Die Anwendung von $\sigma(\cdot)$ auf einen Vektor meint hier Anwendung von $\sigma(\cdot)$ auf jedes Element des Vektors.

Gewichte:

$$\begin{aligned} w_{1,1}^{[1]} &= 1 \\ w_{2,1}^{[1]} &= 2 \\ w_{3,1}^{[1]} &= 3 \\ w_{1,2}^{[1]} &= -1 \\ w_{2,2}^{[1]} &= 4 \\ w_{3,2}^{[1]} &= -2 \\ b_1^{[1]} &= 1 \\ b_2^{[1]} &= 2 \\ b_3^{[1]} &= 0 \end{aligned}$$



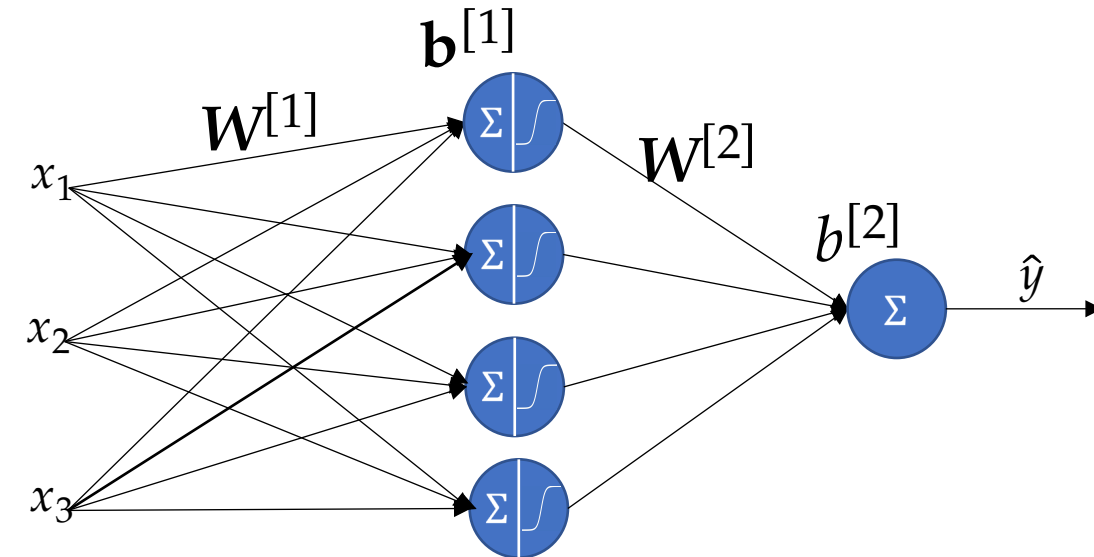
Output verdeckte Schicht:
$$\begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix} = \sigma \left(\begin{pmatrix} w_{1,1}^{[1]} & w_{1,2}^{[1]} \\ w_{2,1}^{[1]} & w_{2,2}^{[1]} \\ w_{3,1}^{[1]} & w_{3,2}^{[1]} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + \begin{pmatrix} b_1^{[1]} \\ b_2^{[1]} \\ b_3^{[1]} \end{pmatrix} \right) = \sigma \left(\mathbf{W}^{[1]} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + \begin{pmatrix} b_1^{[1]} \\ b_2^{[1]} \\ b_3^{[1]} \end{pmatrix} \right)$$

Output Ausgabeschicht:
$$\hat{y} = \begin{pmatrix} w_{1,1}^{[2]} & w_{1,2}^{[2]} & w_{1,3}^{[2]} \end{pmatrix} \begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix} + b_1^{[2]} = \mathbf{W}^{[2]} \begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix} + b^{[2]}$$

Output als geschlossene Formel:
$$\hat{y} = \mathbf{W}^{[2]} \sigma(\mathbf{W}^{[1]} \mathbf{x} + \mathbf{b}^{[1]}) + b^{[2]}$$

Matrixschreibweise

- Die Matrixschreibweise $\hat{y} = \mathbf{W}^{[2]} \sigma(\mathbf{W}^{[1]} \mathbf{x} + \mathbf{b}^{[1]}) + b^{[2]}$ beschreibt *jedes beliebige* einschichtige neuronale Netz, bei dem eine quantitative Vorhersage gemacht wird (**Regression**).
- Dimensionen:
 - $\mathbf{W}^{[1]} \in \mathbb{R}^{m \times p}$: Zeilen: Anzahl Neuronen verdeckte Schicht. Spalten: Anzahl Feature (Eingänge)
 - $\mathbf{b}^{[1]} \in \mathbb{R}^m$: 1 Bias („Schwellwert“) für jedes Neuron der verdeckten Schicht.
 - $\mathbf{W}^{[2]} \in \mathbb{R}^{1 \times m}$: 1 Zeile, da ein Vorhersagewert. Spalten: Anzahl Neuronen verdeckte Schicht.
 - $b^{[2]} \in \mathbb{R}$: 1 Bias für Output-Neuron.



Aktivierungsfunktionen

Neben der Sigmoid-Funktion gibt es weitere Aktivierungsfunktionen, die in neuronalen Netzen zum Einsatz kommen, u.a

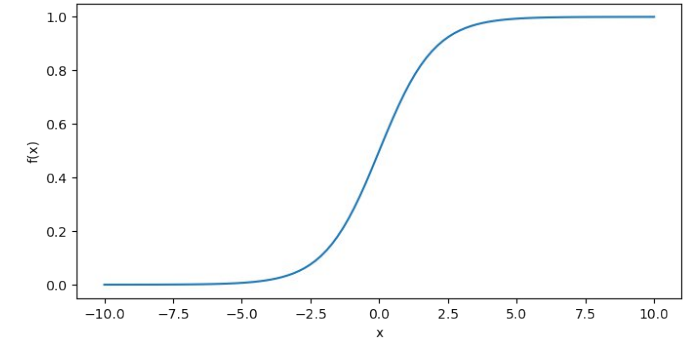
- Tangens Hyperbolicus tanh
- Rectified linear unit, ReLU (heutzutage am meisten verbreitet)

Wichtig: Die Aktivierungsfunktion muss **nichtlinear** sein, sonst würde man insgesamt wieder ein lineares Modell erhalten.

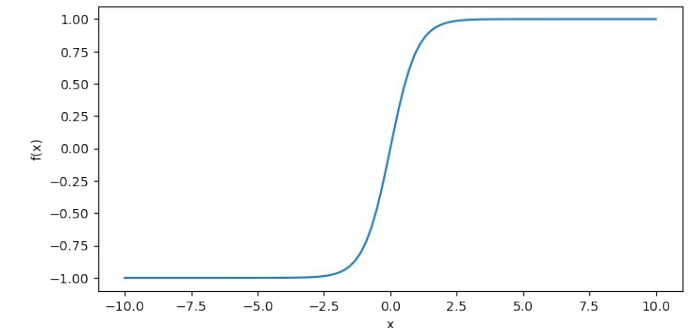
Wir bezeichnen allgemein die Aktivierungsfunktion mit $g(\cdot)$.

Wenn das Argument ein Vektor ist: g wird auf alle Komponenten angewendet.

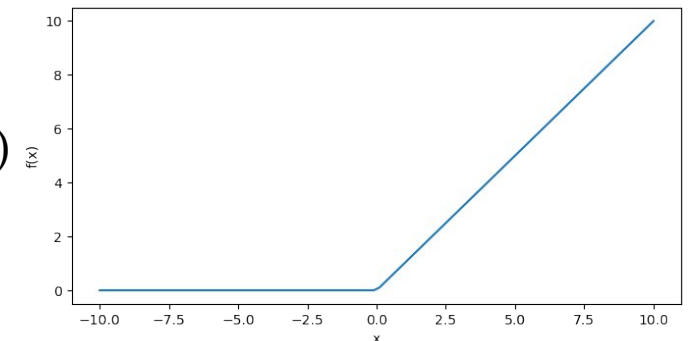
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



$$ReLU(x) = \max(0, x)$$



Neuronale Netze: Mathematische Eigenschaft

Neuronale Netze definieren eine Klasse “universeller Approximatoren”: Resultat von Cybenko (1988) :

Theorem 2. *Let σ be any continuous sigmoidal function. Then finite sums of the form*

$$G(x) = \sum_{j=1}^N \alpha_j \sigma(y_j^T x + \theta_j)$$

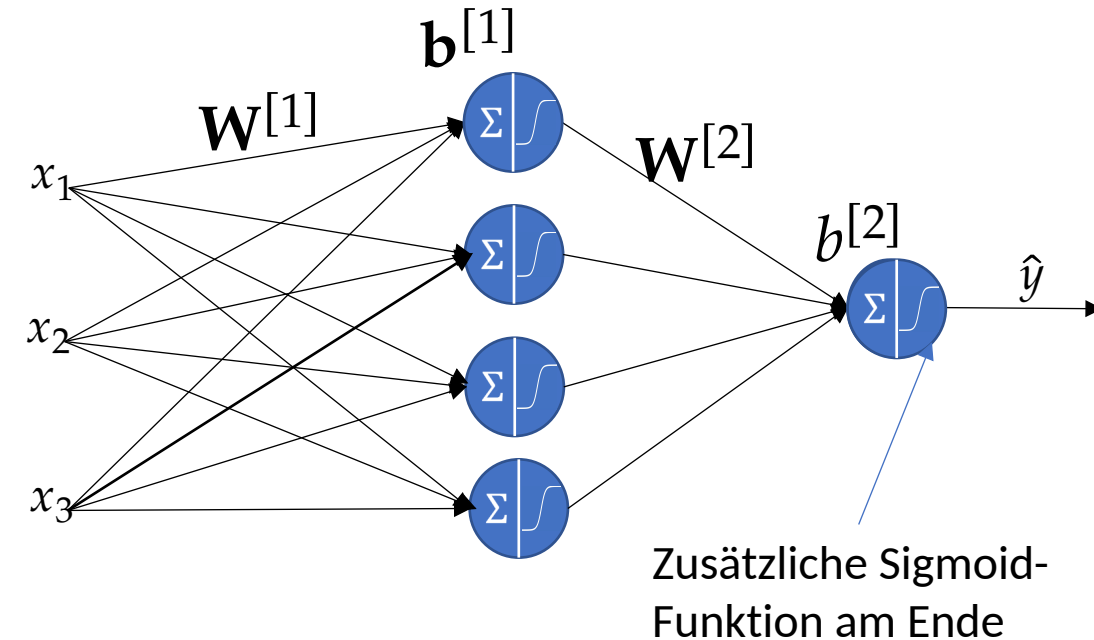
are dense in $C(I_n)$. In other words, given any $f \in C(I_n)$ and $\varepsilon > 0$, there is a sum, $G(x)$, of the above form, for which

$$|G(x) - f(x)| < \varepsilon \quad \text{for all } x \in I_n.$$

Das Theorem stellt sicher, dass sogar ein Netzwerk mit nur einer versteckten Schicht jedes beliebige Klassifikations- oder Regressionsproblem (mit stetiger decision boundary bzw. Modellfunktion) darstellen kann.

Ausgabeschicht Klassifikationsproblem

- Soll statt einer Regression eine (binäre) Klassifikation durchgeführt werden, führt man am Ende noch eine Sigmoid-Funktion aus, um den Output auf $[0,1]$, d.h. eine Wahrscheinlichkeit abzubilden.
- Insgesamt lautet die Vorhersage dann:
$$\hat{y} = \sigma(\mathbf{W}^{[2]} \sigma(\mathbf{W}^{[1]}x + \mathbf{b}^{[1]}) + b_1^{[2]})$$
- Anmerkung: Dies ist eine mehrfache Verkettung von Funktionen (linear + (nichtlineare) Aktivierung)



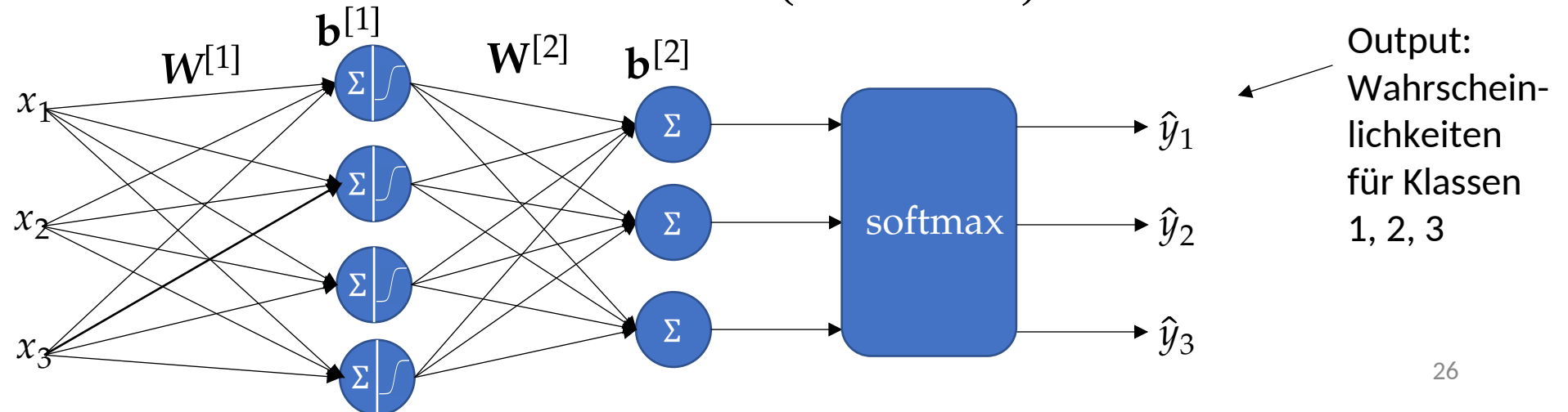
Ausgabeschicht für Mehrklassen-Klassifikationsproblem

- Soll eine Klassifikation in K Klassen durchgeführt werden, hat die Ausgabeschicht K Neuronen (im Bsp $K = 3$). Auf die Werte dieser Neuronen wird am Ende die softmax Funktion

$$\text{softmax}(\mathbf{z}) = \left[\frac{e^{z_1}}{\sum_{j=1}^K e^{z_j}} \quad \frac{e^{z_2}}{\sum_{j=1}^K e^{z_j}} \quad \dots \quad \frac{e^{z_K}}{\sum_{j=1}^K e^{z_j}} \right]$$

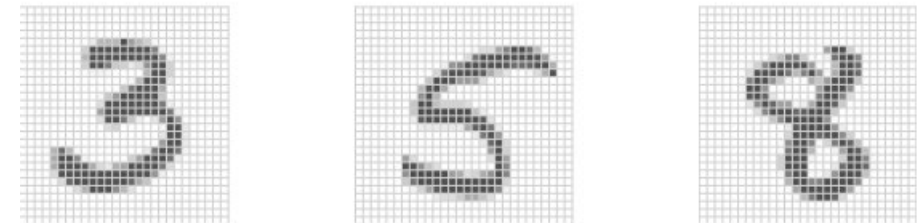
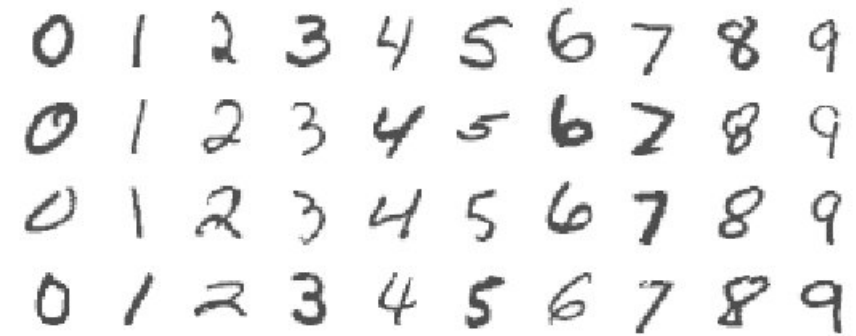
angewendet um diese in eine Wahrscheinlichkeitsverteilung zu transformieren

- Die Dimensionen von $\mathbf{W}^{[2]}$ und $\mathbf{b}^{[2]}$ ändern sich entsprechend: $\mathbf{W}^{[2]} \in \mathbb{R}^{K \times m}$, $\mathbf{b}^{[2]} \in \mathbb{R}^K$
- Insgesamt lautet die Vorhersage dann: $\hat{y} = \text{softmax}(\mathbf{W}^{[2]} \sigma(\mathbf{W}^{[1]} \mathbf{x} + \mathbf{b}^{[1]})) + \mathbf{b}^{[2]}$



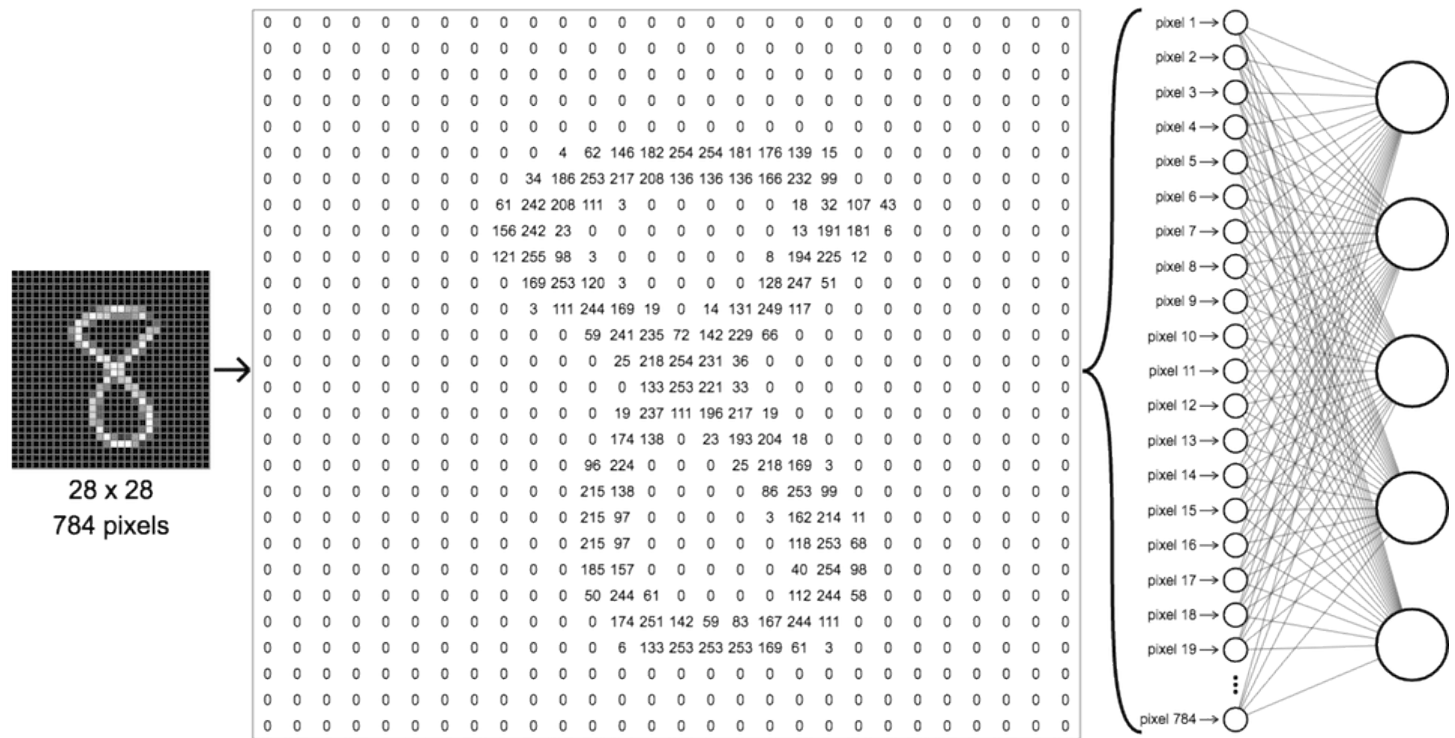
Beispiel: MNIST Datensatz

- Der MNIST Datensatz enthält handgeschriebene Ziffern mit zugehörigen Labels 0,1,...,9
- Größe des Datensatzes: 60,000 Beispiele, 10,000 Beispiele als Testdaten
- Jedes Bild besteht aus 28x28 Pixeln in Graustufen (Wert zwischen 0 und 255)
- Sehr gut untersucht, Standard-Beispiel für neuronale Netze. Original-Quelle: <http://yann.lecun.com/exdb/mnist/>
- Klassifikationsproblem mit $K = 10$ Klassen (Ziffern 0 bis 9)



MNIST Beispiel: Datenrepräsentation

- **Input:** Arrangiere Grauwerte (z.B. spaltenweise) in Vektor der Länge $28 \times 28 = 784$
- **Output:** 10 Klassen werden als 10-dimensionaler Vektor repräsentiert (One-hot-encoding). Das Netz soll für jede Klasse eine Wahrscheinlichkeit vorhersagen.



MNIST Beispiel: Neuronales Netz

Anzahl Parameter:

- Alle 784 Pixel sind mit allen 100 verdeckten Neuronen verbunden:

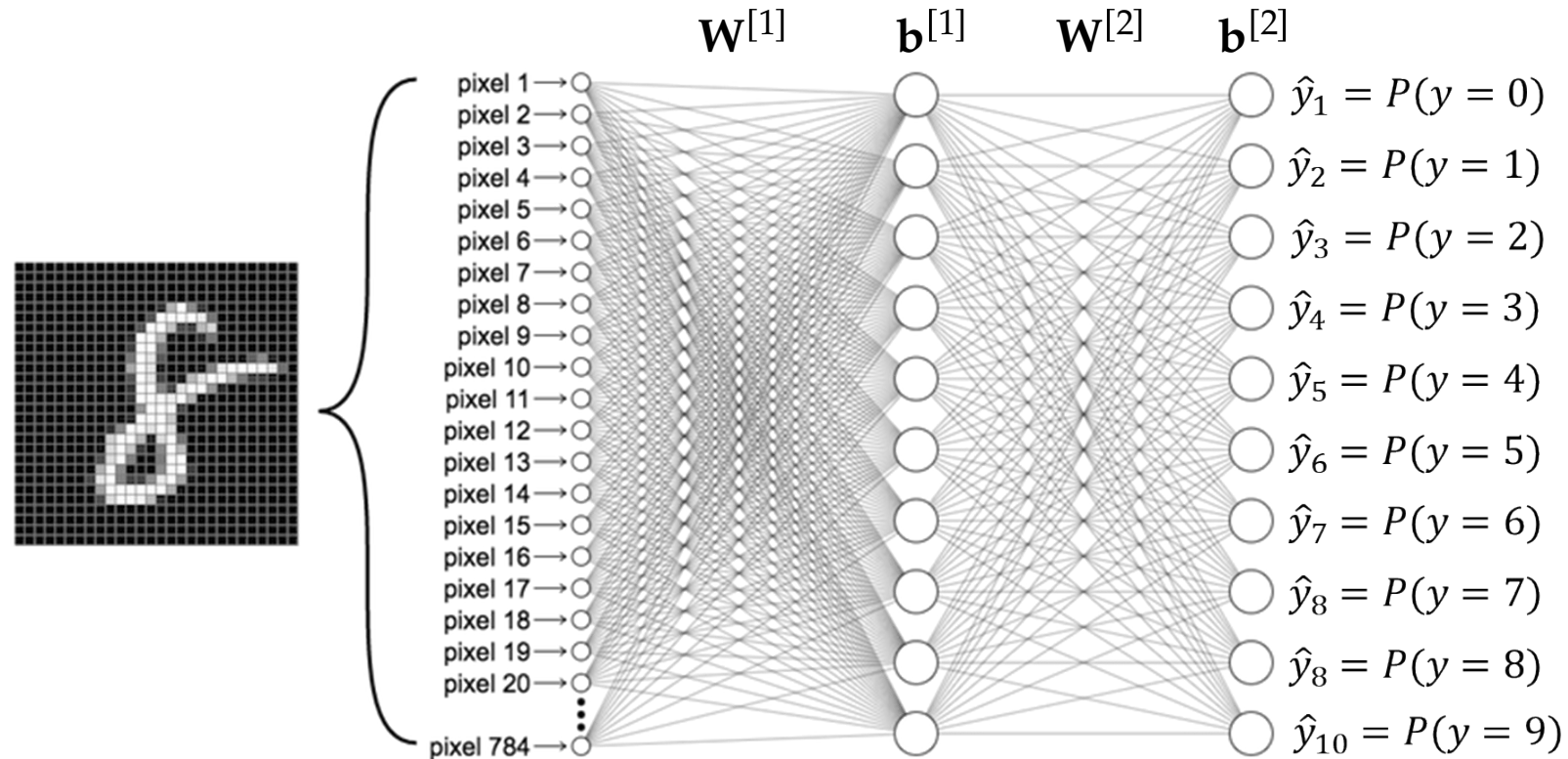
$$\mathbf{W}^{[1]} \in \mathbb{R}^{100 \times 784}, \mathbf{b}^{[1]} \in \mathbb{R}^{100}$$

- 100 verdeckte Neuronen mit 10 Outputwerten:

$$\mathbf{W}^{[2]} \in \mathbb{R}^{10 \times 100}, \mathbf{b}^{[2]} \in \mathbb{R}^{10}$$

Insgesamt 79,510 Parameter, die gelernt werden müssen!

Je mehr verdeckte Neuronen, desto flexibler das Modell und desto mehr Parameter. Aber auch hier: Gefahr von Overfitting.



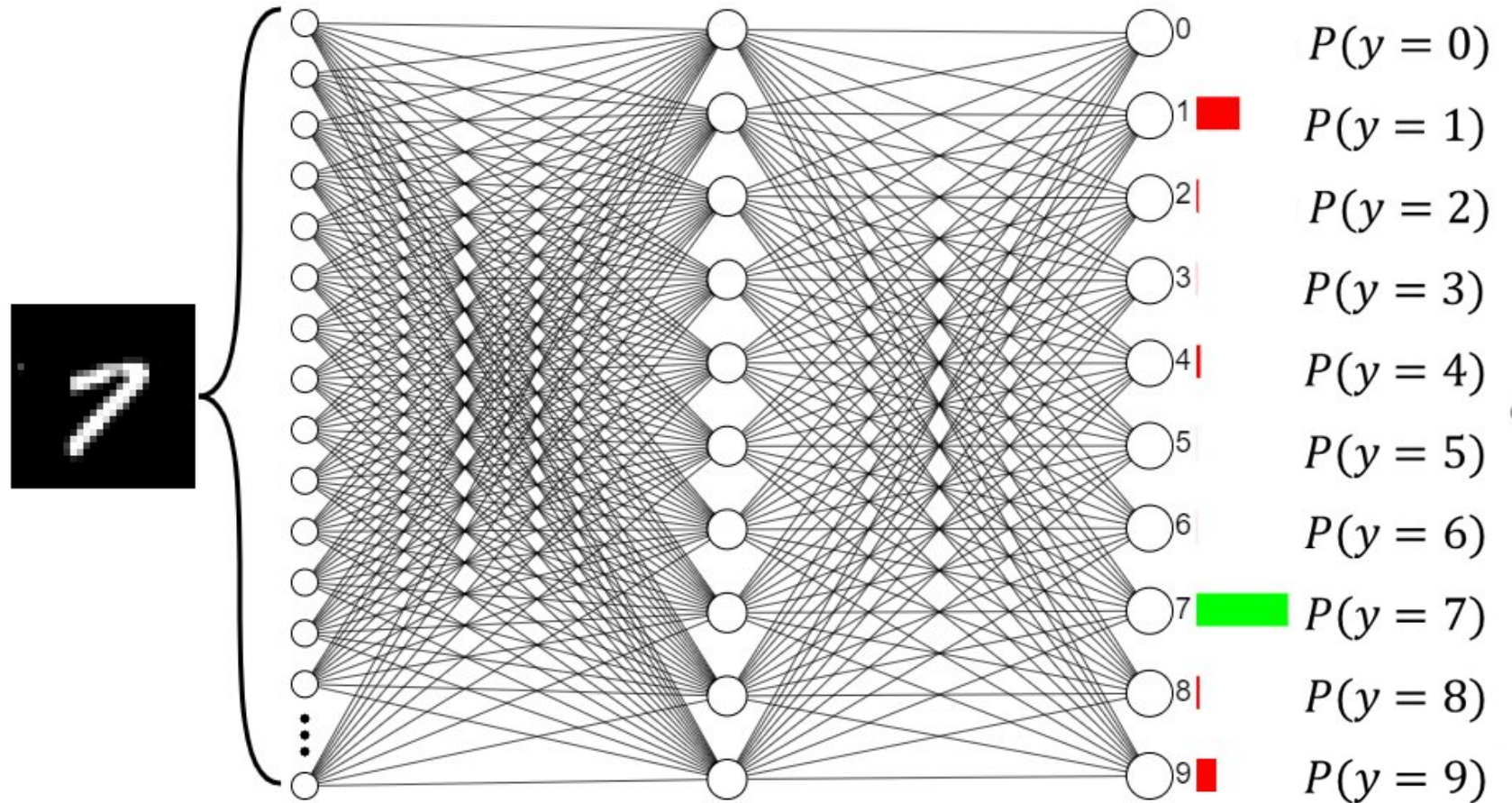
Eingabeschicht:
784 Werte

Verdeckte Schicht:
Bsp: 100 Neuronen
(kann bel. gewählt
werden)

Ausgabeschicht:
10 Werte
(Softmax)

MNIST Beispiel: Vorhersage

- Der Output der letzten Schicht gibt die Wahrscheinlichkeiten für jede Klasse an.
- Es wird diejenige Ziffer vorhergesagt, die durch die Klasse mit der höchsten Wahrscheinlichkeit repräsentiert wird.



MNIST Beispiel: Evaluation des Klassifikators

Für den Klassifikator kann die Accuracy ausgewertet werden:

$$\frac{\# \text{korrekte Vorhersagen}}{\# \text{alle Vorhersagen}}$$

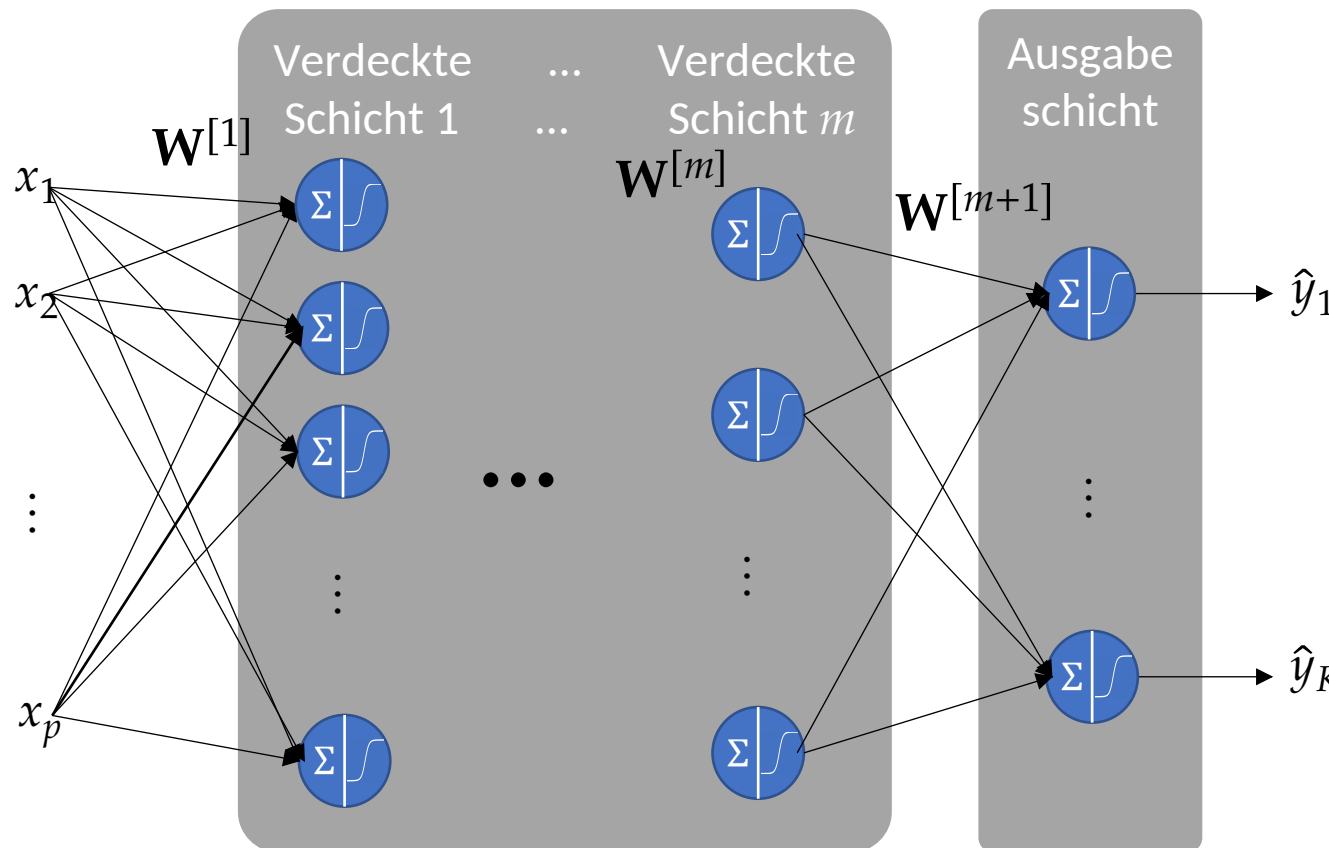
Außerdem kann man die 10x10 Confusion Matrix auswerten.

Precision und recall können für jede der Klassen bestimmt werden

	predicted 0	predicted 1	predicted 2	predicted 3	predicted 4	predicted 5	predicted 6	predicted 7	predicted 8	predicted 9	recall
actual 0	954	0	0	7	1	10	6	3	7	3	96%
actual 1	0	1031	4	3	1	4	1	2	16	2	97%
actual 2	12	21	852	18	11	8	14	20	29	5	86%
actual 3	2	5	9	899	1	71	0	12	23	7	87%
actual 4	2	8	2	2	861	7	7	1	4	89	88%
actual 5	7	5	9	24	3	833	12	8	12	2	91%
actual 6	11	6	2	0	6	31	902	0	8	1	93%
actual 7	3	10	5	3	7	7	1	1041	0	14	95%
actual 8	2	28	4	29	2	31	1	9	882	21	87%
actual 9	7	3	1	7	10	11	1	44	4	873	91%
precision	95%	92%	96%	91%	95%	82%	95%	91%	90%	86%	accuracy 91%

Neuronale Netze mit mehreren Schichten

Man kann beliebig viele Schichten hintereinander schalten, um so besonders expressive Modelle zu erhalten. Allgemeine Form Mehrschicht-Netzwerke:



Die Ausgabewerte der k -ten Schicht sind die Eingabewerte der $k + 1$ -ten Schicht:

$$\begin{aligned} a^{[0]} &= \mathbf{x} \\ \text{Für } k &= 1, \dots, m: \\ \mathbf{a}^{[k]} &= g(\mathbf{W}^{[k]} \mathbf{a}^{[k-1]} + \mathbf{b}^{[k]}) \\ \hat{\mathbf{y}} &= f(\mathbf{W}^{[m+1]} \mathbf{a}^{[m]} + \mathbf{b}^{[m+1]}), \end{aligned}$$

Wobei f die Ausgabefunktion ist, z.B.
 $f(\mathbf{z}) = \mathbf{z}$
 $f(\mathbf{z}) = g(\mathbf{z})$ (Aktivierungsfkt.)
 $f(\mathbf{z}) = \text{softmax}(\mathbf{z})$

Deep Learning

Der Begriff Deep Learning bezeichnet Methoden des maschinellen Lernens, bei denen als Modellfunktion **tiefe** neuronale Netze, d.h. mit mindestens zwei verdeckten Schichten, eingesetzt werden.

Dies kann sich auf unterschiedliche Problemklassen beziehen:

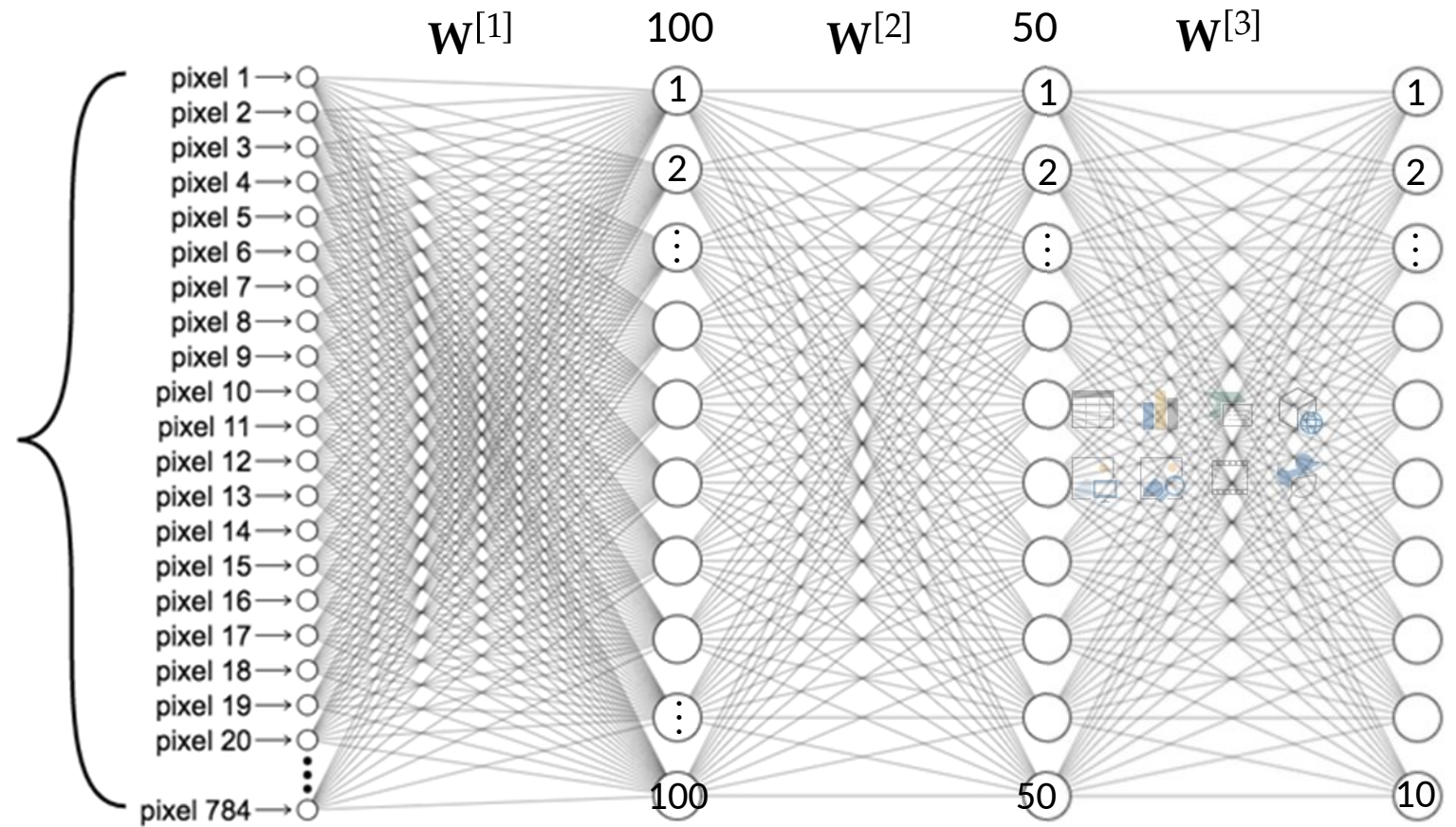
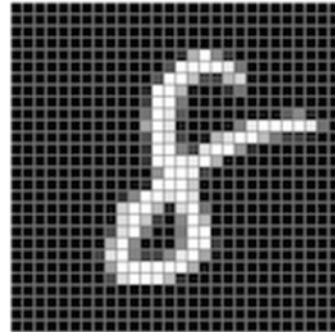
- Überwachtes Lernen (supervised learning)
- Unüberwachtes Lernen (unsupervised learning)
- Bestärkendes Lernen (reinforcement learning)

MNIST Beispiel

Beispiel:

Schicht 1: 100 Neuronen

Schicht 2: 50 Neuronen



$$\mathbf{a}^{[1]} = g\left(\mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}\right), \quad \mathbf{W}^{[1]} \in \mathbb{R}^{100 \times 784}, \mathbf{b}^{[1]} \in \mathbb{R}^{100}, \mathbf{x} \in \mathbb{R}^{784}$$

$$\mathbf{a}^{[2]} = g\left(\mathbf{W}^{[2]}\mathbf{a}^{[1]} + \mathbf{b}^{[2]}\right), \quad \mathbf{W}^{[2]} \in \mathbb{R}^{50 \times 100}, \mathbf{b}^{[2]} \in \mathbb{R}^{50}$$

$$\mathbf{a}^{[3]} = \text{softmax}\left(\mathbf{W}^{[3]}\mathbf{a}^{[2]} + \mathbf{b}^{[3]}\right), \quad \mathbf{W}^{[3]} \in \mathbb{R}^{10 \times 50}, \mathbf{b}^{[3]} \in \mathbb{R}^{10}$$

Insgesamt lautet die Vorhersage für einen Datenpunkt also:

$$\hat{y} = \text{softmax}\left(\mathbf{W}^{[3]} g\left(\mathbf{W}^{[2]} g\left(\mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}\right) + \mathbf{b}^{[2]}\right) + \mathbf{b}^{[3]}\right)$$

—> Vorhersage hängt ab von den Features $\mathbf{x}$ und den (zu lernenden) Parameters $\mathbf{W}^{[k]}, \mathbf{b}^{[k]}$.

Anzahl Parameter: $100 \cdot 784 + 100 + 50 \cdot 100 + 50 + 10 \cdot 50 + 10 = 84,060$

Training von neuronalen Netzen

Wie bisher: Aufstellen und Minimieren einer Verlustfunktion! Formal ordnen wir dafür alle Parameter der einzelnen Schichten (Matrizen $\mathbf{W}^{[k]}$ und Vektoren $\mathbf{b}^{[k]}$) in einem Vektor $\mathbf{w}$ an.

Aufgabe: Finde denjenigen Vektor w , so dass eine geeignete Verlustfunktion minimal wird:

$$\min_{\mathbf{w}} L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \ell(y_i, f(\mathbf{x}_i, \mathbf{w}))$$



Das ist das
neuronale Netz

MNIST Beispiel: Ergebnisse

84,060 Parameter, 60,000 Trainingsbeispiele

Method	Test Error
Neural Network + Ridge Regularization	2.3%
Neural Network + Dropout Regularization	1.8%
Multinomial Logistic Regression	7.2%

Beachte:

Mehr Parameter als Trainingsbeispiele – wäre bei linearen Modellen nicht möglich/bzw. sinnlos

Demo: Visualisierung von neuronalen Netzen

- <http://playground.tensorflow.org/>